Mersenne-Cuccaro Prime Field Arithmetic Circuit Design for Shor's Algorithm

Chiahsun Lee
ScienceVR, Inc. (Palo Alto, USA)
jackie@sciencevr.com

March 2026 v0.2 draft

Abstract

This paper presents a gate-level Shor's algorithm quantum circuit design for Elliptic Curve Discrete Logarithm Problem (ECDLP) over Mersenne prime field built with Cuccaro ripple-carry adders. This design supports circuit construction and simulation tested over curves $y^2 = x^3 + 7$ with Mersenne prime fields $\mathbb{F}_{31}$, $\mathbb{F}_{127}$, $\mathbb{F}_{8191}$, $\mathbb{F}_{131071}$, and $\mathbb{F}_{524287}$ on consumer PCs by exploiting: (1) the structure of Mersenne primes $p = 2^n - 1$ for low-overhead modular reduction via end-around carry, (2) the input-restoration property of Cuccaro adders to reduce arithmetic subroutines, and (3) a sparse state vector simulator for memory-efficient execution of circuits.

Keywords. Quantum computing; Shor's algorithm; ECDLP; Mersenne primes; Cuccaro adder; quantum circuit design; resource estimation.

1 Introduction

Quantum computation can be used to exploit probabilitic structure of natural phenomena. Shor's algorithm for discrete logarithm problem poses quantum threat to applications built with elliptic curve discrete logarithm problem (ECDLP) [1]. While factoring large integers requires substantial fault-tolerant resources [2], the ECDLP variant of Shor's algorithm targets 256-bit ECDSA, ECDH, and related schemes used throughout modern cryptographic infrastructure.

While efforts were made to do resource estimation using Shor's algorithm for solving ECDLP [3], it is still difficult to reproduce results on quantum or classical computers. This motivates a consumer-hardware friendly approach that targets specific reversible prime-field arithmetic to build a true gate-level oracle (rather than only symbolic resource counts), together with an end-to-end simulation path that can be independently verified.

1.1 Contributions

This paper addresses the gap between theoretical resource counts [3, 4] and executable implementation by presenting:

1. **Gate-level, instance-independent ECDLP oracle on consumer hardware:** An end-to-end circuit construction of the controlled ECDLP group action over curves $y^2 = x^3 + 7$ for Mersenne prime fields $\mathbb{F}_{31}$, $\mathbb{F}_{127}$, $\mathbb{F}_{8191}$, $\mathbb{F}_{131071}$, and $\mathbb{F}_{524287}$, using explicit reversible prime-field arithmetic. The oracle depends only on public inputs (G, Q) and requires no secret-dependent table synthesis or lookup.

2. **Mersenne-Cuccaro arithmetic building blocks:** A modular-addition architecture that combines Mersenne end-around carry reduction with Cuccaro ripple-carry adders, together with a consistent reversible "embedding" pattern. For primes $p = 2^n - 1$, addition uses end-around carry (a conditional increment) for modular reduction, and multiplication by powers of two is a cyclic rotation (wire relabeling). This avoids general compare-and-subtract reduction circuits in the Mersenne setting. The Cuccaro ripple-carry adder [5] restores its input operand automatically, eliminating the need for explicit uncomputation in shift-and-add multiplication.

3. **Reproducible resource accounting and executable artifacts:** A circuit generator, gate/depth accounting, and a gate-stream execution path that enables third parties to reproduce the reported qubit/gate counts and perform end-to-end gate-level simulation under a sparse state-vector model for Mersenne primes $p \in \{31, 127, 8191, 131071, 524287\}$ on consumer PCs. The accuracy of secret k recovery is out of the scope of this paper because k-recovery is depending on post-measurement processing pipelines.

2 Shor's ECDLP Oracle Design

Given a base point G on an elliptic curve $E(\mathbb{F}_p)$ and a public point $Q = [k]G$ where k is the secret discrete logarithm, Shor's algorithm recovers k using quantum phase estimation on the group action [6].

In the two-register form, the oracle implements:

$$|a\rangle|b\rangle|P_0\rangle \mapsto |a\rangle|b\rangle|P_0 + [a]G + [b]Q\rangle, \tag{1}$$

where $|a\rangle$ and $|b\rangle$ are phase-estimation (control) registers holding scalars a, b (typically prepared in uniform superposition) and $|P_0\rangle$ is a work register holding an elliptic-curve accumulator point.

This map is unitary because it is a reversible in-place "add-into-target" update of the form $(a, b, P_0) \mapsto (a, b, P_0 + f(a, b))$ with $f(a, b) = [a]G + [b]Q$: the control registers are unchanged, and the transformation can be inverted by subtracting the same group element.

The width t sets the phase-estimation precision (about t bits). Iterative phase-estimation variants can reduce ancilla overhead while retaining comparable precision [7, 8]. However, $2t$ is one of the practical bottlenecks for end-to-end classical simulation because our sparse-state approach scales primarily with the control superposition size ($\sim 2^{2t}$ non-zero amplitudes) and with the compiled gate stream length. In this work we demonstrate end-to-end runs up to $t = 11$ (i.e., $2t = 22$ control qubits) on consumer hardware.

2.1 Qubit Roles and Entanglement Structure

The phase registers ($|a\rangle, |b\rangle$) are the only registers placed in wide superposition; the work register is driven mainly by reversible arithmetic controlled by phase bits.

After control preparation, the controlled oracle typically produces states of the form

$$\sum_{a,b} \alpha_{a,b} \, |a\rangle|b\rangle \, |P_0 + [a]G + [b]Q\rangle, \tag{2}$$

which can induce high entanglement across the control–work bipartition.

2.2 Jacobian Coordinates

To avoid modular inversion inside the quantum circuit, we represent points in Jacobian projective coordinates $(X : Y : Z)$ where the affine point is $(X/Z^2, Y/Z^3)$. The point at infinity is represented by $Z = 0$.

The curve equation becomes:

$$Y^2 = X^3 + aXZ^4 + bZ^6 \pmod{p}. \tag{3}$$

For the secp256k1-family curve $y^2 = x^3 + 7$, the parameter $a = 0$ simplifies point doubling formulas.

2.3 Mersenne Prime Fields

A Mersenne prime has the form $p = 2^n - 1$ for an exponent n such that $2^n - 1$ is prime. This structure provides three critical circuit simplifications (see also generalized Mersenne reduction techniques [9]):

- Modular addition reduces via *end-around carry* rather than compare-and-conditional-subtract-by-p.
- Multiplication by 2^k is a *cyclic rotation* (wire relabeling).
- Negation/subtraction can use *bitwise complement* identities.

End-around carry. Since $2^n \equiv 1 \pmod{p}$, overflow in n-bit addition wraps back as a carry-in to the least significant bit:

$$(a + b) \bmod p = \begin{cases} (a + b) \bmod 2^n & \text{if no overflow} \\ ((a + b) \bmod 2^n) + 1 & \text{if overflow.} \end{cases} \tag{4}$$

Cyclic shifting. Multiplication by 2^k is a cyclic left rotation of k positions—pure wire relabeling with zero gate cost:

$$2^k \cdot x \equiv \mathrm{rotl}_k(x) \pmod{p}. \tag{5}$$

Bitwise complement negation. The additive inverse is the bitwise complement:

$$-x \equiv \bar{x} \pmod{p}, \tag{6}$$

since $x + \bar{x} = 2^n - 1 \equiv 0 \pmod{2^n - 1}$.

2.4 The Cuccaro Adder

The Cuccaro ripple-carry adder [5] computes $|a\rangle|b\rangle \mapsto |a\rangle|a+b\rangle \pmod{2^n}$ using a single work qubit for the ripple and an optional carry-out flag. Its key property is automatic input restoration: the input register $|a\rangle$ is unchanged after the operation.

Cuccaro's construction is built from MAJ/UMA primitives. In the gate-level form, the building blocks on three wires (c, b, a) are:

$$\mathrm{MAJ}(c, b, a): \ \mathrm{CNOT}(a \to b);\ \mathrm{CNOT}(a \to c);\ \mathrm{Toffoli}(c, b \to a), \tag{7}$$

$$\mathrm{UMA}(c, b, a): \ \mathrm{Toffoli}(c, b \to a);\ \mathrm{CNOT}(a \to c);\ \mathrm{CNOT}(c \to b). \tag{8}$$

Gate-counting convention. Throughout this paper, we treat Toffoli (CCNOT) as a primitive gate.

2.5 Mersenne Addition with Cuccaro

We combine Cuccaro addition with Mersenne end-around carry. Instead of building a separate subtraction circuit, this architecture takes the carry-out flag (the overflow bit) from the most significant bit of the Cuccaro ripple-carry adder and literally wraps it around the circuit. This overflow wire is fed directly back into the least significant bit as a carry-in. This acts as a conditional increment: if the addition overflows, it automatically adds 1 to the bottom bit, executing the modulo p reduction in a single, streamlined pass.

Conceptually, it is an out-of-place reversible embedding in the Bennett style: implement the irreversible classical function $f(a, b) = a + b \bmod p$ as a reversible map $(a, b, 0) \mapsto (a, b, f(a, b))$ by allocating an explicit output ("history") register.

Bennett history qubits follow the standard reversible-simulation "compute, copy out, uncompute" pattern attributed to Bennett [10], with later work formalizing time/space trade-offs and reversible pebbling strategies [11].

2.6 Shift-and-Add Multiplication

Field multiplication uses the standard shift-and-add decomposition:

$$a \cdot b = \sum_{i=0}^{n-1} b_i \cdot (a \cdot 2^i) \pmod{p}. \tag{9}$$

For Mersenne fields, each $a \cdot 2^i$ is a cyclic rotation (zero gates). The Cuccaro adder's restoration property means we do not need to copy a before each iteration—it survives the controlled addition unchanged.

2.7 Mixed-Addition Formula

For adding a constant affine point (x_2, y_2) to a quantum Jacobian point (X_1, Y_1, Z_1), we implement the complete addition formula that handles all edge cases. The algorithm computes:

$$\begin{aligned}
U_2 &= x_2 \cdot Z_1^2 \\
S_2 &= y_2 \cdot Z_1^3 \\
H &= U_2 - X_1 \\
R &= S_2 - Y_1 \\
X_3 &= R^2 - H^3 - 2X_1H^2 \\
Y_3 &= R(X_1H^2 - X_3) - Y_1H^3 \\
Z_3 &= H \cdot Z_1.
\end{aligned} \tag{10}$$

These Jacobian–affine mixed-addition formulas follow standard short-Weierstrass references as collected in the Elliptic Curve Formula Database (EFD) [12], and textbook coverage appears in [13, 14].

2.8 Full Shor Circuit Structure

Algorithm (Two-Register Shor ECDLP Circuit).

1. Initialize control registers $|a\rangle = |0\rangle^{\otimes t}$ and $|b\rangle = |0\rangle^{\otimes t}$.
2. Initialize an accumulator point $|P_0\rangle$ (Jacobian representation).
3. Apply $H^{\otimes 2t}$ to the control registers.
4. For each $i = 0, \ldots, t-1$, apply a controlled mixed-addition of the precomputed affine point $[2^i]G$ controlled by a_i.
5. For each $i = 0, \ldots, t-1$, apply a controlled mixed-addition of the precomputed affine point $[2^i]Q$ controlled by b_i.
6. (Simulation strategy) Optionally measure the accumulator/work register and discard it.
7. Apply an inverse QFT (typically in semiclassical/iterative form) to $|a\rangle$ and $|b\rangle$ and measure [15].

3 Sparse State Vector Simulation

3.1 Design Rationale

Our Shor-ECDLP oracle circuits are dominated by classically reversible arithmetic: X, CNOT, Toffoli, and SWAP. On computational basis states, these gates act as permutations of bitstrings; diagonal phase gates multiply amplitudes without changing support. The main source of amplitude splitting is the Hadamard layers on the control registers used for phase estimation (for early quantum-circuit treatments of arithmetic subroutines, see e.g. [16]).

This structure motivates a sparse state representation for oracle validation: after the initial $H^{\otimes 2t}$ on the control registers, the controlled arithmetic subroutines primarily act as controlled permutations on the work bitstrings (rather than broad mixing layers that densify the state). In this regime, memory feasibility is dominated by the control superposition size (roughly 2^{2t}) and by whether mixing gates touch the work register.

3.2 Measurement and Simulation Strategy

To enable end-to-end validation on consumer hardware, our reference implementation employs a virtual measurement strategy: we conceptually measure the work register immediately after the controlled group action and then process the control registers branch-by-branch. This is exact for the marginal measurement distribution on the control registers.

Operationally, the simulator proceeds as:

1. **Collapse:** iterate over non-zero work-register basis states to isolate the corresponding (generally unnormalized) control state.
2. **Controls-only IQFT:** apply the inverse QFT coherently only on the $2t$ control qubits (often using a semiclassical/iterative form) and compute probabilities.
3. **Accumulate:** sum the resulting control-register distributions over all work-register outcomes.

This can be viewed as simulating the control marginal state after tracing out the work register, avoiding the memory blow-up that would occur if one attempted to apply a fully coherent IQFT to a highly entangled control–work state.

4 Resource Estimation Results

4.1 Experimental Setup

The results were produced using the following configuration:

- **System:** Intel Core i7 – Windows 11 – 16 GB DDR4
- **GPU:** NVIDIA GeForce RTX 2070 8GB and 5090 32GB VRAM

4.2 Circuit Building

The following table shows our preliminary non-optimized implementation and results on consumer hardware.

p	A/B Registers	Total Qubits	Total Gates	Circuit Build Time
31 (∼5-bit k)	$5 \times 2 = 10$	296	2,394,810	4.19 sec
127 (∼7-bit k)	$7 \times 2 = 14$	494	6,687,830	11.35 sec
8191 (∼13-bit k)	$10 \times 2 = 20$	1158	37,036,674	66.44 sec
8191 (∼13-bit k)	$11 \times 2 = 22$	1238	40,595,530	74.58 sec
131071 (∼17-bit k)*	$11 \times 2 = 22$	1678	76,175,632	80.66 sec
524287 (∼19-bit k)*	$11 \times 2 = 22$	1910	98,991,802	101.63 sec

Table 1: Circuit and key-recovery statistics for the full unrolled circuits (* indicates larger-field runs on RTX 5090 GPU).

4.3 Circuit Simulation

The following table shows non-optimized gate-level circuit simulation with VRAM usage. On consumer hardware, the main bottlenecks are non-zero states and total gates. The hardware needs to process all non-zero states going through the total gates.

p	Control Qubits t	Non-zero States	Gate Rate	Total Time	Peak VRAM
31 (∼5-bit k)	5	1,024	∼299K/s	9.40 s	∼0.002 GB
127 (∼7-bit k)	7	16,384	∼426K/s	20.31 s	∼0.006 GB
8191 (∼13-bit k)	10	1,048,576	∼62K/s	20 m 18 s	∼0.72 GB
8191 (∼13-bit k)	11	4,194,304	∼16K/s	1 h 12 m	∼2.77 GB
131071 (∼17-bit k)*	11	4,194,304	∼96K/s	3 h 44 m 17 s	∼3.80 GB
524287 (∼19-bit k)*	11	4,194,304	∼94K/s	14 h 5 m 31 s	∼3.11 GB

Table 2: Sparse simulation performance (* indicates larger-field runs on RTX 5090 GPU).

5 Discussion

Can a consumer PC process a 31-bit curve $\mathbb{F}_{2147483647}$ ($2^{31} - 1$ Mersenne prime field)? Probably. Most consumer hardware can process up to 11 control qubits (total 22 phase registers, 2.77GB VRAM shown in Table 3). Increasing the control qubits for larger phase-estimation width t increases

Control Qubits	Total Phase Registers	Non-zero States	State Storage
10	20	1×10^6	∼0.72 GB VRAM
11	22	4×10^6	∼2.77 GB VRAM
12	24	16×10^6	∼9.6 GB VRAM
13	26	67×10^6	∼38.4 GB VRAM/RAM
14	28	268×10^6	∼153.6 GB HDD
15	30	1.07×10^9	∼614.4 GB HDD
16	32	4.29×10^9	∼2.46 TB HDD
17	34	17.2×10^9	∼9.83 TB HDD/Network
18	36	68.7×10^9	∼39.3 TB HDD/Network
19	38	275×10^9	∼157 TB Network

Table 3: Hardware feasibility vs. total control qubits (both phase registers) under the sparse-state scaling assumption. Increasing 1 control qubit requires potentially 4 times more state storage.

the natural control superposition size as 2^{2t}, which in turn drives the sparse state-vector memory footprint to explode expoentially. To effectively recover secret k for 31-bit, you may need around 27 to 30 control-qubits that will require all the computing resources on earth. However, k recovery accuracy is out of the scope of this paper.

For Mersenne prime fields such as $\mathbb{F}_{31}$, $\mathbb{F}_{127}$, $\mathbb{F}_{8191}$, $\mathbb{F}_{131071}$, and $\mathbb{F}_{524287}$, we demonstrate end-to-end Shor ECDLP oracle construction and simulation. The current implementation prioritizes simplicity and reproducibility over qubit minimization. Peak work-qubit counts remain high due to retained Bennett history registers.

Several directions remain for optimization and extension, including qubit-minimizing schedules via reversible pebbling, alternative MCX decompositions, k-recovery accuracy, iterative phase estimation, post-measurement processing, and extension to other prime forms such as pseudo-Mersenne/Solinas primes ($2^n - c$) instead of Mersenne primes ($2^n - 1$). All of above become incrementally feasible based on this working implementation.

References

[1] Peter W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE, 1994.

[2] Craig Gidney and Martin Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021.

[3] Martin Roetteler, Michael Naehrig, Krysta M. Svore, and Kristin Lauter. Quantum resource estimates for computing elliptic curve discrete logarithms. In *ASIACRYPT 2017*, pages 241–270, 2017.

[4] Thomas Häner, Samuel Jaques, Michael Naehrig, Martin Roetteler, and Mathias Soeken. Improved quantum circuits for elliptic curve discrete logarithms. In *PQCrypto 2020*, pages 425–444, 2020.

[5] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. A new quantum ripple-carry addition circuit. arXiv:quant-ph/0410184, 2004.

[6] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

[7] Alexei Yu. Kitaev. Quantum measurements and the Abelian stabilizer problem, 1995.

[8] Mikuláš Dobšíček, Goran Johansson, Vitaly Shumeiko, and Gunter Wendin. Arbitrary accuracy iterative quantum phase estimation algorithm using a single ancillary qubit. *Physical Review A*, 76(3):030306, 2007.

[9] Jerome A. Solinas. Generalized mersenne numbers. Tech. rep. corr 99-39, Centre for Applied Cryptographic Research, University of Waterloo, 1999.

[10] Charles H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973.

[11] Charles H. Bennett. Time/space trade-offs for reversible computation. *SIAM Journal on Computing*, 18(4):766–776, 1989.

[12] Daniel J. Bernstein and Tanja Lange. Elliptic curve formula database (efd). `https://www.hyperelliptic.org/EFD/`. Accessed 2026.

[13] Darryl Hankerson, Alfred J. Menezes, and Scott Vanstone. *Guide to Elliptic Curve Cryptography*. Springer, 2004.

[14] Henri Cohen and Gerhard Frey, editors. *Handbook of Elliptic and Hyperelliptic Curve Cryptography*. Chapman and Hall/CRC, 2005.

[15] Robert B. Griffiths and Chi-Sheng Niu. Semiclassical fourier transform for quantum computation. *Physical Review Letters*, 76(17):3228–3231, 1996.

[16] Vlatko Vedral, Adriano Barenco, and Artur Ekert. Quantum networks for elementary arithmetic operations. *Physical Review A*, 54(1):147–153, 1996.